PART 1

Data Analysis

Turning measured audio into 57 features and a clean learning table.

SIGNALS AND SYSTEMS · MACHINE LEARNING

From Audio Signal to Genre Prediction

How extracted audio features become a defensible classification decision.

Music Genre Classification using Signal Processing and Machine Learning

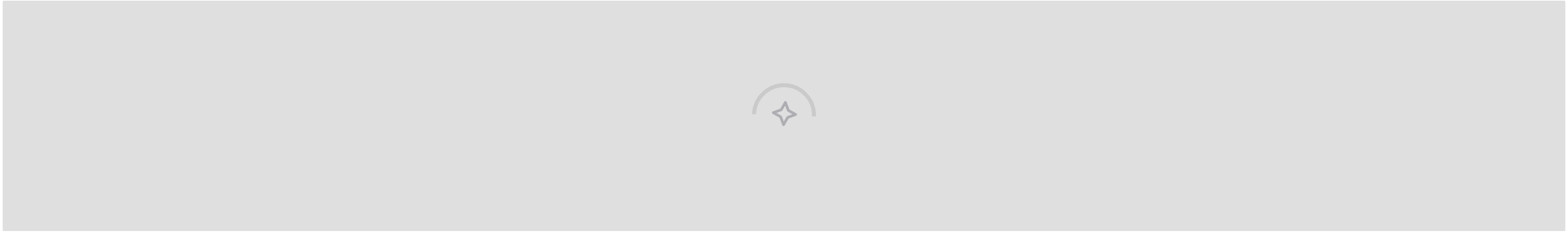
From measured sound to a usable decision · University project readout

PROJECT LENS

10 genres · 57 features · song-level evaluation

Signals and Systems Still Drive the Result

Machine Learning never sees the original waveform directly. It learns from descriptors computed after sampling, segmentation, and time-frequency analysis.



STFT

Tracks frequency content over time.

MFCCs

Summarize perceived spectral shape and timbre.

Chroma

Captures energy across pitch classes.

Tempo

Adds pulse and rhythmic evidence.

Spectral features

Describe brightness, spread, and rolloff.

Takeaway: Signal processing supplies the measurements; machine learning supplies the decision boundary.

The Dataset Turns Music into a Learning Table

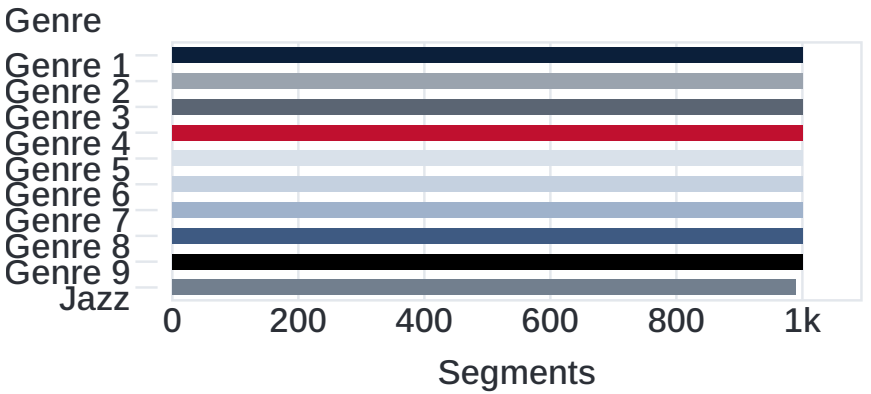
Before training, the notebook checks what the model can use, what must be excluded, and whether the data is complete enough to trust.

TABLE STRUCTURE

Field group	What it contains	Model use
Identifiers	song_id, label	Excluded from X
Descriptor	57 numerical audio features	Input matrix X
Metadata	length	Excluded from X
Target	10 genre labels	Prediction y

9,981 rows · 60 columns · zero missing values

SEGMENTS PER GENRE



Approximately balanced; jazz is slightly smaller.

Notebook code

```
df = pd.read_csv("music_features.csv")
df.shape
df["label"].value_counts()
df.isnull().sum()
```

Takeaway: Dataset inspection prevents the model from learning from the wrong columns or hidden data-quality problems.

What One Row Represents

Each row of the feature table is one 3-second clip, not one song. Rows from the same song_id share a recording.

SEGMENT → SONG

Song jazz.00009.wav	**Segments** 3-second clips
Aggregation mean and variance per clip	**Row** 57 features + label

Source: uploaded PDF, df.sample(5) output (length = 66150 per row)

WHY SEGMENTS ARE NOT INDEPENDENT

- song_id (e.g., jazz.00009.wav) groups all clips from one recording
- Clips from one song share tempo, timbre, and recording conditions
- Training on one clip and testing on its sibling inflates accuracy

Teaching cue: Ask what the unit of analysis is. If the answer is “a segment”, ask what happens when one song is split across train and test.

Source: uploaded PDF, notebook df.shape = (9981, 60)

A2 · FEATURE VECTOR

The 57-Feature Vector

After dropping song_id, label, and length, X has 57 numeric columns: 17 descriptors with mean and variance, plus tempo, plus 20 MFCC pairs.

TIMBRE AND SPECTRUM · 12

chroma_stft, rms, spectral_centroid, spectral_bandwidth, rolloff, zero_crossing_rate: mean and var each

HARMONY AND PERCUSSION · 4

harmony and perceptr: mean and var each

TEMPO · 1

tempo

MFCC · 40

mfcc1 to mfcc20: mean and var each

Teaching cue: For each family, ask which signal property it represents: loudness (RMS), brightness (spectral centroid), pitch class (chroma), rhythm (tempo), or timbre envelope (MFCC).

Source: uploaded PDF, X.shape = (9981, 57)

Exploration Shows What the Features Look Like

Before choosing a model, the notebook visualizes the 57 audio features to reveal scale differences, skew, genre overlap, correlations, and possible outliers for preprocessing.

Distributions & shape

Genre distribution: The bar chart is close to balanced, so accuracy is not dominated by one large class.

Histograms: Feature values have different ranges and shapes; scaling and transformation may be needed before fitting.

KDE plots: Density curves show unimodal, bimodal, and multimodal behavior; skewed or multimodal inputs can make a simple linear boundary less effective.

Genre spread, dependencies & outliers

Boxplots: Genre groups overlap, but some features separate genres better; long whiskers mark unusual observations that may distort the learned boundary.

Heatmap: Correlations show related descriptors with overlapping information; correlated or extreme features can affect stability, so preprocessing is tested next.

Largest outlier shares: harmony_mean 22.04%, perceptr_mean 15.34%, rms_var 9.17%.

Notebook code removes non-feature columns, plots distributions, and visualizes feature correlations:

```
features = df.drop(columns=["song_id", "label", "length"])
features.hist(bins=30)
sns.heatmap(features.corr(), cmap="coolwarm", center=0)
plt.show()
```

Takeaway: Exploration does not classify the songs; it shows how carefully features must be prepared before classification.

PART 2

Feature Engineering and Logistic Regression

Preventing leakage, scaling safely, and building the linear baseline.

Why Data Leakage Inflates Accuracy

Segments from one song are near-duplicates. A random split lets the model recognize songs it has already seen.

RANDOM ROW SPLIT (RISKY)

Clip 1 of jazz.00009 goes to train; clip 2 of the same song goes to test.

The test score then rewards memorized recordings.

Illustrative; the notebook did not run a random split.

GROUP-SAFE SPLIT (NOTEBOOK)

All clips of a song stay on one side of the split.

Songs in both sets: 0

```
groups = df["song_id"]
X = df.drop(columns=["label", "song_id", "length"])
train_idx, test_idx = next(sgkf.split(X, y, groups))
print("Songs in both sets:", len(set(groups.iloc[train_idx]) & set(groups.iloc[test_idx])))
```

Teaching cue: song_id is a group label, never a predictor. Using it as a feature would reward memorizing recordings rather than learning genre.

Source: uploaded PDF, notebook split check

Preprocessing Prevents the Model from Cheating

The split must reflect the real task: unseen songs, not merely unseen segments. Learn transformations on training data only to prevent leakage.

57 features	Group-safe split	Transform	Expand	Model
Drop identifiers	Separate songs	Scale, reshape, clip	Optional interactions	Predict genre

GROUP-SAFE SPLIT RESULT

7,992 training rows

1,989 test rows

0 songs shared across the split

```
X = df.drop(columns=["label", "song_id", "length"])
y = df["label"]
groups = df["song_id"]
sgkf = StratifiedGroupKFold(n_splits=5, shuffle=True,
random_state=42)
train_idx, test_idx = next(sgkf.split(X, y, groups))
```

WHY EACH STEP EXISTS

- **Inputs:** X drops label, song_id, and length, leaving 57 numerical features; y is the genre target, and groups uses song_id.
- **StratifiedGroupKFold:** preserves genre proportions and keeps each song on one side; the split gives a more realistic test on unseen songs.
- **StandardScaler + PowerTransformer:** learn common mean and spread from training data, then reshape skewed distributions for a more stable boundary.
- **Clipping:** limits extreme values to the 1st–99th percentile range before transforming.
- **Polynomial Features:** adds squared and pairwise terms for interactions among timbre, rhythm, and spectral features.

Takeaway: A believable score requires clean, comparable, leakage-free evidence.

A4 · VALIDATION

How StratifiedGroupKFold Works

Stratification keeps genre proportions, groups keep each song whole, and each fold takes a turn as the test block.

FIVE FOLDS · ONE SHOWN

Fold 1	Folds 2–5	Index
test	train	train_idx, test_idx

NOTEBOOK SPLIT RESULT

Set	Rows	Shared songs
Train	7,992	0
Test	1,989	0

Test genres: 190–200 rows each (jazz 190, hiphop 199)

KEY TERMS

- Stratified: genre mix kept in every fold
- Group: song_id never crosses a boundary
- Fold: one block held out for testing

Validation question: Why does a single group-safe split still vary, and why report the mean across five folds?

Source: uploaded PDF, notebook split output

Scaling, Skew, and Robustness

Features have very different ranges and skew, so every transform is learned from training data only.

STANDARDSCALER

$$z = \frac{x - \mu_{\text{train}}}{\sigma_{\text{train}}}$$

Test accuracy 0.647

POWERTRANSFORMER

Power map that reduces skew toward a more bell-shaped feature.

Test accuracy 0.651

PERCENTILE CLIPPING

Clip each feature to its training 1st–99th percentile, then power-transform.

Test accuracy 0.659

```
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)  # learn mean and std from train
X_test_scaled = scaler.transform(X_test)        # reuse train statistics
```

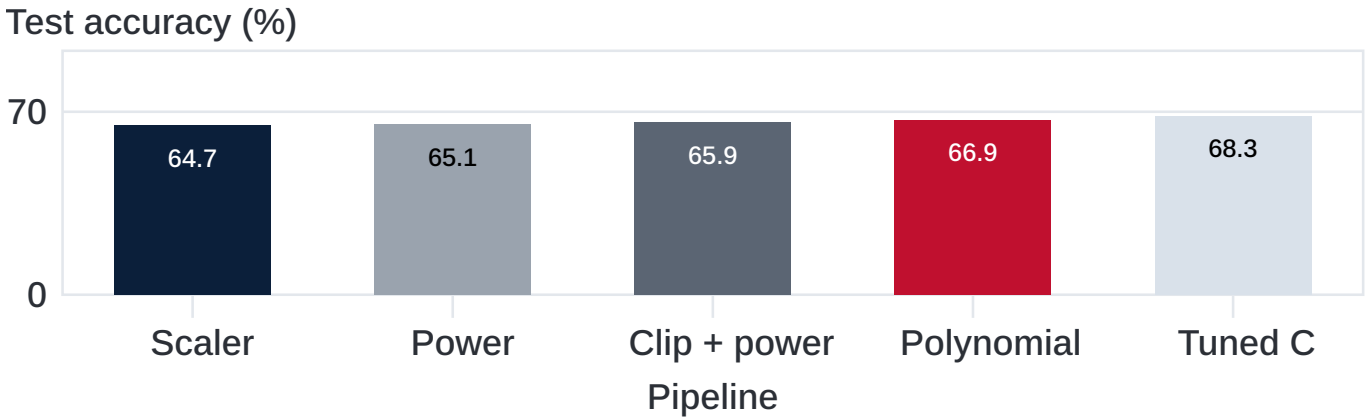
Teaching cue: Ask why fit_transform on the test set would leak test-set statistics into the model.

Source: uploaded PDF, notebook preprocessing results

Logistic Regression Builds the Baseline

Logistic Regression learns weighted combinations of the features and converts them into genre probabilities. Its transparency makes it the right reference before adding nonlinear flexibility.

TEST ACCURACY BY PIPELINE



Test accuracy rises as representation improves, but complexity also raises overfitting pressure.

WHAT THE NUMBERS SAY

- **StandardScaler:** 75.2% training, 64.7% test.
- **PowerTransformer:** test accuracy reaches 65.1%.
- **Clipping + PowerTransformer:** reaches 65.9%.
- **Polynomial expansion:** reaches 66.9%; tuning C reaches 68.3%.
- **Warning:** 100% training accuracy signals overfitting pressure.

```
model = LogisticRegression(max_iter=2000)
model.fit(X_train_poly, y_train)
test_pred = model.predict(X_test_poly)
accuracy_score(y_test, test_pred)
```

Takeaway: Better preparation helps, but mostly linear boundaries remain limited.

Polynomial Features and Regularization

Degree-2 terms let a linear model use feature interactions. The parameter C sets how strongly it is held back from overfitting.

COMPLEXITY MAP

Underfit

Linear, train 0.752 · test 0.647

Overfit

Degree 2, C=1: train 1.000 · test 0.669

57 features expand to 1,710 (linear, squared, and pairwise terms).

C COMPARISON (DEGREE 2)

C	Train	Test
0.0005	0.838	0.667
0.002	0.918	0.680
0.01	0.980	0.683

Teaching cue: Ask why training accuracy of 1.0 is a warning sign, not a success, and which C value the evidence supports.

Source: uploaded PDF, notebook C comparison

PART 3

Support Vector Machine

Nonlinear boundaries, tuning, and defending the final model.

RBF SVM Learns Nonlinear Genre Boundaries

A Support Vector Machine finds a separating surface with a margin. Its radial basis function kernel allows curved regions where genres overlap in the 57-feature space.

MODEL INTUITION

Margin

A buffer around the boundary supports generalization.

RBF kernel

Similarity creates flexible, curved decision regions.

C and gamma

C controls error pressure; gamma controls locality.

TUNING GRID

C	Gamma tested	Selected
3	0.0175 · 0.03 · 0.06 · 0.1	C=3
10	0.0175 · 0.03 · 0.06 · 0.1	Not selected

GridSearchCV tests combinations using song-separated validation.

SEARCH SETUP

Stage	Configuration
Pipeline	PowerTransformer() → SVC(kernel="rbf")
C values	3, 10
Gamma values	0.0175, 0.03, 0.06, 0.1
Validation	GridSearchCV with StratifiedGroupKFold(...)

Notebook result: initial SVM gives 72.2% segment accuracy and 76.4% song accuracy; the tuned setting is C=3, gamma=0.03.

A7 · KERNEL METHOD

Inside the RBF SVM

The SVM keeps only support vectors near the boundary, and the RBF kernel lets that boundary bend around the genres.

SUPPORT VECTORS

Training clips closest to the boundary. They define the margin.

HYPERPLANE AND KERNEL

A flat cut in a higher-dimensional space appears as a curved region in the original 57-feature space.

C AND GAMMA

C trades margin width against errors. gamma sets how far one support vector reaches.

```
final = make_pipeline(PowerTransformer(), SVC(kernel="rbf", C=3, gamma=0.03))
final.fit(X_train, y_train)
```

Teaching cue: Connect the curved regions to overlapping genres. Rock, disco, hiphop, and reggae share rhythmic and timbral traits, so boundaries there are hardest to draw.

Source: uploaded PDF, notebook SVM results

What GridSearchCV Actually Does

The search scores every candidate with grouped cross-validation on training data only, then refits the best one.

HOW IT WORKS

- Parameter grid: C in {3, 10}, gamma in {0.0175, 0.03, 0.06, 0.1}
- 8 candidates × 3 group folds = 24 fits
- mean_test_score = mean validation accuracy
- best_params_ = C 3, gamma 0.03

COMPACT SEARCH TABLE

C	gamma	Mean CV
3	0.030	0.7187
10	0.030	0.7175
3	0.0175	0.7173
10	0.0175	0.7168
3	0.100	0.6197

Teaching cue: Ask why the test set was not used to pick C or gamma, and how close the top candidates are: C=3 leads C=10 by only about 0.001.

Source: uploaded PDF, notebook GridSearchCV output

Song-Level Evaluation Matches the Real Task

The model predicts short segments, but the application needs one label per song. Aggregating segment evidence makes the evaluation match that real decision.

Segments

Each 3-second window gets scores.

Group

song_id reconnects windows.

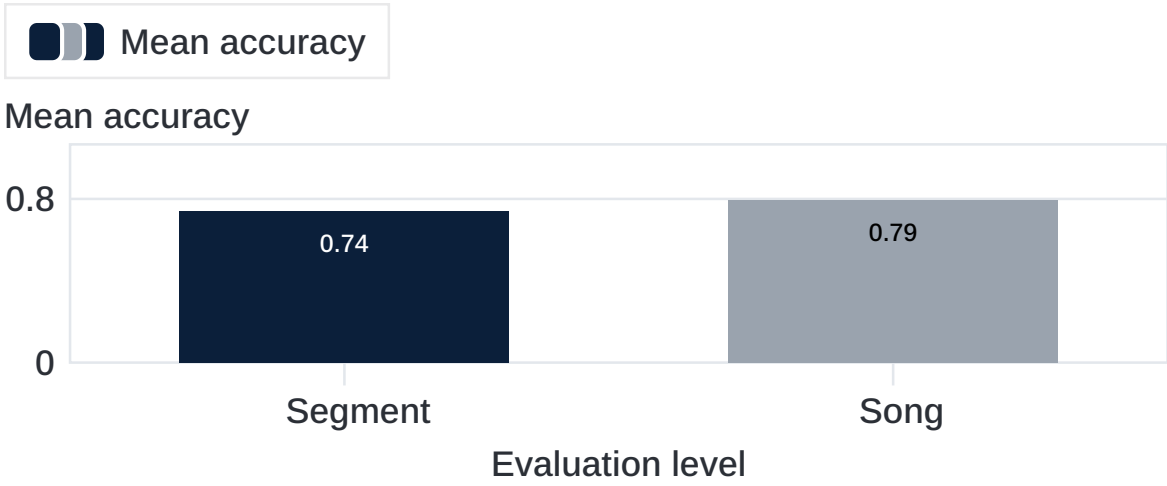
Average

Evidence is combined by song.

Decide

Highest combined score wins.

FIVE-FOLD PERFORMANCE



Segment: 0.741 ± 0.018 · Song: 0.794 ± 0.029

```
scores["song"] = groups.iloc[test_idx].values
song_pred = scores.groupby("song").mean().idxmax(axis=1)
song_accuracy = accuracy_score(song_true, song_pred)
```

Takeaway: Repeated evidence across a song produces a more stable final genre decision than one isolated segment.

Reading the Confusion Matrix

Rows are true genres and columns are predictions. Diagonal cells are correct song-level calls; off-diagonal cells are confusions.

PRECISION, RECALL, F1 (SONG LEVEL)

Genre	Prec.	Recall	F1
Classical	0.90	0.95	0.93
Jazz	0.94	0.84	0.89
Hiphop	0.69	0.55	0.61
Reggae	0.58	0.55	0.56
Rock	0.73	0.40	0.52

MAIN CONFUSIONS

- Hiphop → reggae: 5 songs
- Reggae → hiphop: 4 songs
- Rock → country, disco, metal: 3 each
- Jazz → classical: 2 songs

Song-level SVM, 199 test songs; accuracy 0.76

Teaching cue: Read the confusions as musical overlap. Hiphop and reggae share rhythm, and rock has high precision but low recall, so many rock songs are predicted as other genres.

Source: uploaded PDF, notebook classification report and confusion matrix

Logistic Regression vs. SVM

The comparison is not only about one accuracy number. It balances performance, speed, interpretability, nonlinear behavior, and fit with the song-level task.

Criterion	Logistic Regression	RBF SVM	Project reading
Accuracy	68.3% segment after polynomial expansion	72.7% segment and 76.4% song before tuning; tuned means: 74.1% segment and 79.4% song across five grouped folds	SVM leads at song level
Nonlinear behavior	Mostly linear boundary	RBF models nonlinear boundaries	Better for feature overlap
Speed	Faster	More computationally demanding	Trade-off accepted
Interpretability	Weights are easier to inspect	Less direct	Baseline remains useful
Overfitting risk	Higher after expansion	Controlled through C and gamma	Validate by groups
Project fit	PCA: retaining all 57 features gave the best tested cross-validation score	Polynomial-kernel SVM remained below tuned RBF	Tuned RBF selected

SELECTED MODEL

Tuned RBF SVM, C=3 and gamma=0.03. It gives the strongest grouped validation performance under the same leakage-safe design.

Takeaway: Select the model that best matches the feature geometry and the real evaluation task.

How to Defend the Final Model

Every claim traces back to the group-safe split and the notebook's own evidence.

DEFENSE FLOW

57 features

PowerTransformer

fit on train

RBF SVM

C=3, gamma=0.03

CHECKLIST

- Group-safe validation: 0 shared songs
- Segment 0.741 ± 0.018 vs song 0.794 ± 0.029
- RBF SVM 0.727 vs best logistic model 0.683 (segment)
- Limits: one 80/20 test split, 199 test songs, overlapping genres
- Future: nested CV, more songs, wider feature study

TEACHER QUESTIONS

- Why is song accuracy higher than segment accuracy?
- Why does PCA with 10 components drop to 0.598?
- Why is rock recall only 0.40?
- What would change if songs leaked across folds?

Source: uploaded PDF, notebook results

CONCLUSION

From Measured Sound to a Defensible Model

Reliable genre classification comes from the complete chain: meaningful features, disciplined preprocessing, leakage-safe validation, and a model matched to the feature geometry.

01

Check the table

9,981 rows, 57 usable features, no missing values.

02

Protect the split

Keep songs separated across training and testing.

03

Prepare evidence

Transform, clip, scale, and test interactions.

04

Compare fairly

Use a transparent baseline and grouped validation.

05

Aggregate the result

Evaluate one genre decision per song.

Thank You